CYBER SECURITY
PRACTICAL CODES

Complete Python programs with import libraries – Practicals 1, 2, 4, 5, 6, 7 and 9

Instructions: Run each practical separately in Google Colab or Jupyter Notebook. Each program includes its own import statements. If a library is missing, install it using `pip install numpy pandas matplotlib scikit-learn`.

Note: Practicals 4 and 5 have the same title and implementation in the supplied practical document, so one shared code is included for both.

Practical 1 – Installation Verification (Hello World)

Aim: To verify Python and Jupyter Notebook installation.

Complete Python Code

```
# Practical 1: Hello World

# Import library
import sys

# Display Python version
print("Python Version:", sys.version)

# Hello World program
print("Hello World")
print("Welcome to Python Programming")
```

Expected result

Python Version: (Your installed Python version) Hello World Welcome to Python Programming

Practical 2 – Simple Malware Detection using Decision Tree

Aim: To detect whether a file is Benign or Malware using a Decision Tree classifier.

Complete Python Code

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Create sample dataset
data = {
    'file_size': [120, 150, 180, 200, 5000, 6000, 7000, 8000, 130, 170, 5500, 7500],
    'suspicious_calls': [1, 2, 1, 2, 15, 18, 20, 25, 1, 2, 16, 22],
    'entropy': [3.2, 3.5, 3.1, 3.8, 7.5, 7.8, 7.9, 8.0, 3.4, 3.6, 7.6, 7.8],
    'malware': [0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1]
}

df = pd.DataFrame(data)
print("Dataset:")
print(df)

# Separate features and target
X = df[['file_size', 'suspicious_calls', 'entropy']]
y = df['malware']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Create and train Decision Tree
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Evaluate model
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Predict a new file
new_file = pd.DataFrame(
    [[6500, 20, 7.8]],
    columns=['file_size', 'suspicious_calls', 'entropy']
)

prediction = model.predict(new_file)

if prediction[0] == 1:
    print("\nNew File: Malware Detected")
else:
    print("\nNew File: Benign File")

# Visualize Decision Tree
plt.figure(figsize=(12, 6))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['Benign', 'Malware'],
    filled=True
)
plt.title("Decision Tree Malware Detection")
plt.show()
```

Expected result

Accuracy, confusion matrix, classification report, predicted file class, and a Decision Tree plot.

Practical 4 and 5 – Network Intrusion Detection using Decision Tree and Isolation Forest

Aim: To detect normal and attack network traffic using Decision Tree and Isolation Forest algorithms.

Complete Python Code

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import IsolationForest
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Generate normal network traffic
np.random.seed(42)

normal = pd.DataFrame({
    'duration': np.random.normal(2, 0.5, 500),
    'bytes_sent': np.random.normal(500, 100, 500),
    'connections': np.random.normal(10, 2, 500),
    'label': 0
})

# Generate attack network traffic
attack = pd.DataFrame({
    'duration': np.random.normal(10, 2, 25),
    'bytes_sent': np.random.normal(5000, 500, 25),
    'connections': np.random.normal(120, 15, 25),
    'label': 1
})

# Combine datasets
df = pd.concat([normal, attack], ignore_index=True)

print("Dataset:")
print(df.head())
print("\nClass Distribution:")
print(df['label'].value_counts())

# Visualize network traffic
plt.figure(figsize=(8, 5))
plt.scatter(
    normal['bytes_sent'], normal['connections'],
    label='Normal Traffic'
)
plt.scatter(
    attack['bytes_sent'], attack['connections'],
    label='Attack Traffic'
)
plt.xlabel("Bytes Sent")
plt.ylabel("Connections")
plt.title("Network Traffic Visualization")
plt.legend()
plt.show()

# Separate features and target
X = df[['duration', 'bytes_sent', 'connections']]
y = df['label']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Decision Tree Model
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)

# Prediction
y_pred = dt_model.predict(X_test)

# Evaluation
print("\nDECISION TREE RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred, zero_division=0))

# Isolation Forest Model
iso_model = IsolationForest(
    contamination=25/525,
    random_state=42
)

iso_model.fit(X)

# Predict anomalies
iso_pred = iso_model.predict(X)

# Convert output: 1 = Normal, -1 = Anomaly
iso_pred = np.where(iso_pred == -1, 1, 0)

print("\nISOLATION FOREST RESULTS")
print("Confusion Matrix:")
print(confusion_matrix(y, iso_pred))
print("\nClassification Report:")
print(classification_report(y, iso_pred, zero_division=0))

# Visualize Isolation Forest results
plt.figure(figsize=(8, 5))
plt.scatter(
    X['bytes_sent'],
    X['connections'],
    c=iso_pred
)
plt.xlabel("Bytes Sent")
plt.ylabel("Connections")
plt.title("Isolation Forest Anomaly Detection")
plt.show()
```

Expected result

Decision Tree accuracy, confusion matrix and classification report; Isolation Forest confusion matrix, classification report and anomaly-detection plot.

Practical 6 – User Authentication using K-Nearest Neighbors (KNN)

Aim: To authenticate genuine users and impostors using typing behaviour.

Complete Python Code

```
# Import required libraries
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Generate sample data
np.random.seed(42)

# Genuine user data
genuine = pd.DataFrame({
    'typing_speed': np.random.normal(5.5, 0.3, 200),
    'key_hold_time': np.random.normal(0.12, 0.02, 200),
    'key_gap_time': np.random.normal(0.18, 0.03, 200),
    'label': 1
})

# Impostor data
impostor = pd.DataFrame({
    'typing_speed': np.random.normal(4.2, 0.5, 200),
    'key_hold_time': np.random.normal(0.22, 0.04, 200),
    'key_gap_time': np.random.normal(0.30, 0.05, 200),
    'label': 0
})

# Combine dataset
df = pd.concat([genuine, impostor], ignore_index=True)

print("Dataset:")
print(df.head())

# Separate features and labels
X = df[['typing_speed', 'key_hold_time', 'key_gap_time']]
y = df['label']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Train model
knn.fit(X_train, y_train)

# Predict test data
y_pred = knn.predict(X_test)

# Evaluate model
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Test genuine user
genuine_user = pd.DataFrame(
    [[5.5, 0.12, 0.18]],
    columns=['typing_speed', 'key_hold_time', 'key_gap_time']
)
genuine_user = scaler.transform(genuine_user)
prediction = knn.predict(genuine_user)

if prediction[0] == 1:
    print("\nGenuine User: Access Granted")
else:
    print("\nGenuine User: Access Denied")
```

```
# Test impostor
impostor_user = pd.DataFrame(
    [[3.8, 0.25, 0.35]],
    columns=['typing_speed', 'key_hold_time', 'key_gap_time']
)
impostor_user = scaler.transform(impostor_user)
prediction = knn.predict(impostor_user)

if prediction[0] == 1:
    print("\nImpostor: Access Granted")
else:
    print("\nImpostor: Access Denied")
```

Expected result

Accuracy, confusion matrix, classification report, genuine-user access decision, and impostor access decision.

Practical 7 – Fraud Detection using Logistic Regression and Random Forest

Aim: To detect fraudulent transactions and compare Logistic Regression and Random Forest models.

Complete Python Code

```
# Import required libraries
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)

# Generate sample dataset
np.random.seed(42)
n = 2000

# Generate transaction features
amount = np.random.uniform(100, 10000, n)
time = np.random.uniform(0, 24, n)
location_risk = np.random.uniform(0, 1, n)
device_risk = np.random.uniform(0, 1, n)
num_transactions = np.random.randint(1, 20, n)
balance_change = np.random.uniform(0, 10000, n)

# Generate fraud labels
fraud_score = (
    (location_risk > 0.75).astype(int) +
    (device_risk > 0.75).astype(int) +
    (amount > 8000).astype(int) +
    (time < 5).astype(int)
)
fraud = (fraud_score >= 2).astype(int)

# Create DataFrame
df = pd.DataFrame({
    'amount': amount,
    'time': time,
    'location_risk': location_risk,
    'device_risk': device_risk,
    'num_transactions': num_transactions,
    'balance_change': balance_change,
    'Class': fraud
})

print("Dataset:")
print(df.head())
print("\nClass Distribution:")
print(df['Class'].value_counts())

# Separate features and target
X = df.drop('Class', axis=1)
y = df['Class']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Scale the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Logistic Regression Model
lr = LogisticRegression(class_weight='balanced', random_state=42)
lr.fit(X_train_scaled, y_train)
lr_pred = lr.predict(X_test_scaled)

print("\nLOGISTIC REGRESSION RESULTS")
print("Accuracy:", accuracy_score(y_test, lr_pred))
print("Precision:", precision_score(y_test, lr_pred, zero_division=0))
print("Recall:", recall_score(y_test, lr_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, lr_pred, zero_division=0))
```

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, lr_pred))
print("\nClassification Report:")
print(classification_report(y_test, lr_pred, zero_division=0))

# Random Forest Model
rf = RandomForestClassifier(
    n_estimators=100,
    class_weight='balanced',
    random_state=42
)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("\nRANDOM FOREST RESULTS")
print("Accuracy:", accuracy_score(y_test, rf_pred))
print("Precision:", precision_score(y_test, rf_pred, zero_division=0))
print("Recall:", recall_score(y_test, rf_pred, zero_division=0))
print("F1 Score:", f1_score(y_test, rf_pred, zero_division=0))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_pred))
print("\nClassification Report:")
print(classification_report(y_test, rf_pred, zero_division=0))

# Compare both models
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [
        accuracy_score(y_test, lr_pred),
        accuracy_score(y_test, rf_pred)
    ]
})

print("\nMODEL COMPARISON")
print(comparison)
```

Expected result

Logistic Regression and Random Forest metrics, confusion matrices, classification reports and model-accuracy comparison.

Practical 9 – Evaluating Detector Performance using Feature Engineering

Aim: To improve fraud detection performance by adding engineered features and comparing model results before and after feature engineering.

Complete Python Code

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Generate sample dataset
np.random.seed(42)
n = 5000

# Generate original features
amount = np.random.uniform(100, 10000, n)
avg_amount = np.random.uniform(500, 5000, n)
hour = np.random.randint(0, 24, n)
distance_km = np.random.uniform(1, 500, n)
txn_last_24h = np.random.randint(1, 15, n)

# Generate fraud labels
fraud_score = (
    (amount > avg_amount * 2).astype(int) +
    (hour < 6).astype(int) +
    (distance_km > 300).astype(int) +
    (txn_last_24h > 10).astype(int)
)
fraud = (fraud_score >= 2).astype(int)

# Create DataFrame
df = pd.DataFrame({
    'amount': amount,
    'avg_amount': avg_amount,
    'hour': hour,
    'distance_km': distance_km,
    'txn_last_24h': txn_last_24h,
    'Class': fraud
})

print("Original Dataset:")
print(df.head())
print("\nClass Distribution:")
print(df['Class'].value_counts())

# Feature engineering
df['amount_ratio'] = df['amount'] / (df['avg_amount'] + 1)
df['is_night'] = (df['hour'] < 6).astype(int)
df['log_amount'] = np.log1p(df['amount'])
df['distance_x_txn'] = df['distance_km'] * df['txn_last_24h']

print("\nDataset After Feature Engineering:")
print(df.head())

# Define evaluation function
def evaluate_model(X, y):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20,
        random_state=42,
        stratify=y
    )

    # Standardize data
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    # Train Logistic Regression
    model = LogisticRegression(
        class_weight='balanced',
        max_iter=1000,
        random_state=42
    )
    model.fit(X_train, y_train)
```

```
    # Prediction
    y_pred = model.predict(X_test)

    # Calculate performance metrics
    return {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred, zero_division=0),
        'F1 Score': f1_score(y_test, y_pred, zero_division=0)
    }

# Original five features
original_features = [
    'amount',
    'avg_amount',
    'hour',
    'distance_km',
    'txn_last_24h'
]
X_before = df[original_features]
y = df['Class']

# Evaluate before feature engineering
before = evaluate_model(X_before, y)
print("\nBEFORE FEATURE ENGINEERING")
print(before)

# Original + engineered features
all_features = [
    'amount',
    'avg_amount',
    'hour',
    'distance_km',
    'txn_last_24h',
    'amount_ratio',
    'is_night',
    'log_amount',
    'distance_x_txn'
]
X_after = df[all_features]

# Evaluate after feature engineering
after = evaluate_model(X_after, y)
print("\nAFTER FEATURE ENGINEERING")
print(after)

# Compare results
comparison = pd.DataFrame({
    'Before': before,
    'After': after
})
print("\nPERFORMANCE COMPARISON")
print(comparison)

# Plot comparison graph
comparison.plot(kind='bar', figsize=(10, 6))
plt.title("Before and After Feature Engineering")
plt.xlabel("Performance Metrics")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.legend()
plt.tight_layout()
plt.show()
```

Expected result

Before-and-after accuracy, precision, recall and F1-score comparison, with a bar graph.